

UNITÉ 2 · SEMAINE 2

Appels système et E/S de bas niveau

📅 Semaine 2 · dates à confirmer </> Laboratoire TD1 (corrigé + défi IA) ~ 4 animations

MANUEL M2 · THÉORIE DE L'UNITÉ

Appels système et E/S de bas niveau

Lecture de base de l'unité 2 (≈ 1 h 30). Chaque section se termine par un lien **Pratiquez** → vers l'exercice ou l'animation qui l'illustre. Les chiffres proviennent d'exécutions réelles des programmes du TD1 sur la machine du cours (Linux 7.0, x86-64, glibc 2.43).

SOMMAIRE

- | | | | |
|----|---|----|---|
| §1 | Mode utilisateur et mode noyau | §4 | E/S de bas niveau avec <code>read</code> et <code>write</code> |
| §2 | Anatomie d'un appel système | §5 | Combien coûte un appel |
| §3 | Observer: <code>man</code> , <code>strace</code> , <code>objdump</code> | §6 | Au-delà: <code>vDSO</code> , <code>io_uring</code> , autres architectures |

§1 Mode utilisateur et mode noyau

Le processeur a au moins deux **niveaux de privilège**. Sur x86-64, on les appelle des anneaux: le noyau s'exécute dans l'anneau 0 (CPL 0) et les programmes dans l'anneau 3 (CPL 3). En mode utilisateur, les instructions qui touchent au matériel (entrées/sorties, table des pages, désactivation des interruptions) sont interdites: si un programme les tente, le processeur lève une exception et le noyau le termine.

Cette frontière est le fondement de tout le reste: grâce à elle, un programme ne peut ni lire la mémoire d'un autre, ni écrire sur le disque en contournant les permissions, ni s'approprier le processeur. Quand un programme a besoin de quelque chose que seul le noyau peut faire, **il le demande**: c'est un appel système.

§2 Anatomie d'un appel système

Quand votre programme écrit `write(1, "hola\n", 5)`, il appelle en réalité une fonction de la glibc (l'*enveloppe*) qui suit la convention de Linux x86-64:

1. Elle place le **numéro de l'appel** dans le registre `rax` (1 pour `write`, 0 pour `read`, 110 pour `getppid`) et les arguments dans `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`. Les données ne voyagent pas dans les registres: `rsi` porte l'adresse du texte.
2. Elle exécute l'instruction `syscall`: le processeur passe en mode noyau et saute à une adresse fixe que le noyau a configurée au démarrage (`entry_SYSCALL_64`). Le programme ne peut pas choisir où sauter.

3. Le noyau utilise `rax` comme indice dans `sys_call_table` et exécute le gestionnaire (`ksys_write`), qui valide le descripteur, copie les données depuis la mémoire du programme (`copy_from_user`) et appelle le pilote du périphérique.
4. Le résultat revient dans `rax` et l'instruction `sysret` remet le processeur en mode utilisateur.
5. Si `rax` est entre -4095 et -1, l'enveloppe le traite comme une erreur: elle range le code dans `errno` et renvoie -1. C'est pourquoi le motif de tout le cours est `exit_if(rc == -1, "...")`, qui affiche l'erreur avec `perror`.

Sortie réelle

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -A4 -E '^[0-9a-f]+ <getppid@>'
000000000000ff710 <getppid@GLIBC_2.2.5>:
    ff710:    endbr64
    ff714:    mov     $0x6e,%eax
    ff719:    syscall
    ff71b:    ret
```

Pratiquez → animation A2.1 · Le voyage d'un appel système

§3 Observer: `man`, `strace`, `objdump`

Trois outils pour examiner n'importe quel programme sans son code source:

- `man 2 write` documente l'appel système; `man 3 printf`, la fonction de bibliothèque; `man 3p`, la norme POSIX.
- `strace ./programme` montre chaque appel système avec ses arguments et son résultat; `-f` suit les enfants, `-r` ajoute le temps entre appels et `-c` fait un résumé.
- `objdump -d` désassemble: c'est ainsi qu'on voit l'instruction `syscall` dans la lib.

Une surprise en utilisant `strace`: l'écho du TDI fait une **trentaine d'appels avant d'arriver à `main`**. Ils viennent du chargeur dynamique, qui ouvre `libc.so.6`, la projette en mémoire avec `mmap` et protège ses pages avec `mprotect`.

Pratiquez → animation A2.2 · strace en direct

§4 E/S de bas niveau avec `read` et `write`

Chaque processus naît avec trois **descripteurs de fichier** ouverts: 0 (entrée standard), 1 (sortie standard) et 2 (erreur standard). C'est le shell qui décide vers quoi ils pointent: `< fichier`, `> fichier`, `2>&1` ou un tube. Le programme ne le sait pas: il n'utilise que les numéros.

APPEL	RENVOL	ATTENTION À
<code>read(fd, buf, n)</code>	octets lus (1...n), 0 en fin de fichier, -1 en cas d'erreur	il peut en renvoyer <i>moins</i> que <code>n</code> (tube, terminal, réseau, signal)
<code>write(fd, buf, n)</code>	octets écrits (1...n), -1 en cas d'erreur	écriture partielle (tube ou socket plein); <code>EINTR</code> si un signal l'interrompt

C'est pourquoi les programmes du cours répètent: `read_exact` lit jusqu'à ce que la demande soit complète et `write_all` écrit jusqu'à ce que tout soit sorti. De plus, `read` et `write` copient des **octets**, pas du texte: écrire un `long` copie ses 8 octets dans l'ordre de la machine (*little-endian* sur x86-64), et écrire une structure copie aussi son remplissage d'alignement (24 octets pour `struct nepad`).

Sortie réelle

```
$ ./write_number | od -A d -t x1
00000000 05 00 00 00 00 00 00 00 08 00 00 00 00 00 00
0000016
```

⚠ Bogue classique: avancer le pointeur de ce qui reste

L'exemple de lecture des diapositives d'origine (B25, diapositive 51) faisait `length = length - byte; buf = buf + length;`. Avec un fichier ordinaire, cela marche — la première lecture ramène tout et `length` vaut 0 —, mais avec un tube ou un socket cela laisse des trous et écrit **hors du tampon**. La bonne façon est `buf += byte`, en réessayant sur `EINTR` et en s'arrêtant sur 0. Voyez-le pas à pas dans A2.4 et essayez-le avec `cat file | ./read_struct`.

Pratiquez → TD1, exercices 3 à 7

§5 Combien coûte un appel

Chaque aller-retour vers le noyau a un coût fixe: changer de mode, sauvegarder et restaurer les registres, valider les arguments. Le programme `bench_read` du TD1 le mesure en copiant 8 Mio vers `/dev/null` avec des tampons de 1 octet à 1 Mio. Voici les résultats réels sur la machine du cours:

Sortie réelle

```
$ ./bench_read      # 8 MiB file, buffers of 1 B to 1 MiB
buffer_bytes,read_calls,seconds,MB_per_s,ns_per_iteration
1,8388609,6.4984,1.3,775
4,2097153,1.5997,5.2,763
16,524289,0.3952,21.2,754
64,131073,0.0998,84.1,761
256,32769,0.0260,322.6,794
1024,8193,0.0075,1121.4,913
4096,2049,0.0025,3351.2,1222
16384,513,0.0010,8319.5,1966
65536,129,0.0008,10834.7,6002
262144,33,0.0007,11832.2,21484
1048576,9,0.0008,10287.9,90598
```

Avec 1 octet par appel, il y a 8,4 millions d'itérations et la copie prend des secondes ; avec 64 Kio, 129 itérations et moins d'une milliseconde. Ajuster le modèle $\text{temps} = \text{appels} \times c_0 + \text{octets} \times c_1$ aux mesures donne un péage d'environ **0,7 μs par itération** (`read` + `write`) et moins de 0,1 ns par octet copié. Conclusion pratique : utilisez des tampons de quelques Kio ou plus (la bibliothèque C le fait pour vous avec `printf` / `fwrite`, unité 4) et ne lisez jamais octet par octet dans une boucle.

Pratiquez → animation A2.3 · Le coût d'un appel système

§6 Au-delà : vDSO, `io_uring` et autres architectures

- **vDSO.** Certains « appels » très fréquents, comme `clock_gettime`, n'entrent pas dans le noyau : Linux projette dans chaque processus une petite bibliothèque (vDSO) qui lit l'heure dans une page partagée. C'est pourquoi ils n'apparaissent pas dans `strace`.
- **`io_uring`** (Linux 5.1, 2019). Pour les programmes qui font beaucoup d'E/S : le programme et le noyau partagent deux files en mémoire ; on peut soumettre des centaines d'opérations en un seul appel, voire aucun. La même idée que le gros tampon, appliquée aux appels.
- **Autres architectures.** Sur ARM 64 bits, l'instruction est `svc #0` et le numéro va dans `x8` ; sur RISC-V, c'est `ecall` avec le numéro dans `a7`. Les numéros changent d'une architecture à l'autre ; l'idée, non.
- **Observabilité avec eBPF.** Des outils comme `bpfftrace` comptent les appels dans tout le système pour un coût bien moindre que `strace`, qui arrête le processus à chaque appel.